

One-Pager Framework 활용 가이드

이 문서는 One-Pager의 아키텍처를 범용 프레임워크로 활용하는 방법을 설명합니다. 예시로 **논문 작성 플랫폼**에 적용하는 시나리오를 다룹니다.

1. 프레임워크 구조 개요

One-Pager Framework			
Data Fetcher	AI Pipeline (Generate + Refine)	Template Engine	Interaction Layer
외부 API → 구조화	프롬프트 → 섹션별 스트림	4종 렌더링 → PDF	채팅 → 부분 수정 폴더/버전 관리

핵심 패턴 4가지

패턴	역할	주식 리포트 예시
섹션 기반 데이터 모델	문서를 독립적 섹션으로 분리	header, valuation, thesis, verdict
Progressive Fill	섹션이 하나씩 스트림으로 채워지는 UX	SSE로 각 섹션 순차 렌더링 + 애니메이션
AI 채팅 리파인	대화로 특정 섹션만 수정	"bull case 3번 수정해줘" → thesis만 업데이트
템플릿 스위칭	동일 데이터, 다른 렌더링	Newspaper / Modern / Executive / Compact

2. 적용 방법: 3단계

Step 1: 데이터 모델 정의

프레임워크의 핵심은 `ReportData` 인터페이스입니다. 용도에 맞게 섹션을 재정의합니다.

`lib/types.ts` → 섹션 인터페이스 정의
`lib/prompts.ts` → AI에게 섹션 구조 알려주기

Step 2: Data Fetcher 교체

`lib/market-data.ts`에 해당하는 부분을 교체합니다. 외부 API에서 초기 데이터를 수집하는 로직입니다.

`lib/market-data.ts` → 외부 소스에서 원시 데이터 수집
`api/reports/generate/route.ts` → 수집 → AI 분석 → DB 저장 파이프라인

Step 3: 템플릿 작성

각 템플릿은 순수 React 컴포넌트입니다. `useSections()` 훅으로 데이터를 받아 렌더링합니다.

`app/components/templates/{Name}Template.tsx` → 렌더링
`app/components/templates/shared.tsx` → 공통 유틸 + 타입

나머지는 그대로 동작합니다:

- 폴더/리포트 관리 (Supabase CRUD)

- 버전 관리 (스냅샷 저장/복원)
- AI 채팅 리파인 (섹션 단위 수정)
- PDF 내보내기 (html2canvas + jsPDF)

3. 예시: 논문 작성 플랫폼

3-1. 데이터 모델

```
// lib/types.ts

interface PaperMetadata {
  title: string
  authors: string[]
  affiliation: string
  journal?: string
  keywords: string[]
  submissionDate?: string
}

interface PaperAbstract {
  background: string // 연구 배경
  objective: string // 연구 목적
  method: string // 방법론 요약
  results: string // 주요 결과
  conclusion: string // 결론 한 줄
}

interface PaperIntroduction {
  problemStatement: string
  researchGap: string
  contribution: string[]
  outline: string
}

interface PaperMethodology {
  approach: string
  dataCollection: string
  analysisMethod: string
  limitations: string[]
}

interface PaperResults {
  findings: { label: string; value: string; description: string }[]
  tables: { title: string; headers: string[]; rows: string[][] }[]
  charts: { title: string; data: { name: string; value: number }[] }[]
}

interface PaperDiscussion {
  interpretation: string
  comparison: string // 선행연구 비교
  implications: string[] // 시사점
  futureWork: string[] // 향후 연구 방향
}

interface PaperReferences {
```

```

    items: { id: string; citation: string; doi?: string }[]
  }

interface PaperLayout {
  style?: 'ieee' | 'apa' | 'chicago'
  columns?: 1 | 2
  showCharts?: boolean
  abstractStyle?: 'structured' | 'narrative'
}

// 최종 문서 데이터
interface PaperData {
  metadata: PaperMetadata
  abstract: PaperAbstract
  introduction: PaperIntroduction
  methodology: PaperMethodology
  results: PaperResults
  discussion: PaperDiscussion
  references: PaperReferences
  layout?: PaperLayout
}

```

3-2. Data Fetcher

논문의 경우 외부 데이터 소스가 다양합니다:

```

// lib/paper-data.ts

export async function fetchResearchContext(topic: string) {
  // 1. Semantic Scholar API → 관련 논문 검색
  const relatedPapers = await fetch(
    `https://api.semanticscholar.org/graph/v1/paper/search?query=${topic}&limit=10`
  ).then(r => r.json())

  // 2. 키워드 기반 연구 트렌드
  const trends = extractTrends(relatedPapers)

  // 3. 인용 네트워크 분석
  const citationGraph = buildCitationGraph(relatedPapers)

  return {
    relatedPapers, // AI가 참고문헌으로 활용
    trends,        // 연구 동향 파악용
    citationGraph, // 핵심 논문 식별용
  }
}

```

3-3. AI 프롬프트

```

// lib/prompts.ts

function buildPaperPrompt(topic: string, context: ResearchContext): string {
  return `You are an academic research assistant. Draft a paper on: ${topic}`
}

RELATED PAPERS:

```

```
`${JSON.stringify(context.relatedPapers, null, 2)}`
```

Generate in this JSON format:

```
{
  "metadata": { "title": "...", "keywords": [...] },
  "abstract": { "background": "...", "objective": "...", ... },
  "introduction": { "problemStatement": "...", ... },
  "methodology": { ... },
  "results": { "findings": [...], "tables": [...] },
  "discussion": { "interpretation": "...", ... },
  "references": { "items": [...] }
}
```

Rules:

- Academic tone, formal language
- Cite related papers where relevant
- Return ONLY valid JSON`

```
function buildPaperChatPrompt(paperData, chatHistory, message): string {
  return `You are refining an academic paper. Current data:
  ${JSON.stringify(paperData, null, 2)}`
}
```

EDITABLE SECTIONS:

- "metadata": { title, authors, affiliation, journal, keywords, submissionDate }
- "abstract": { background, objective, method, results, conclusion }
- "introduction": { problemStatement, researchGap, contribution[], outline }
- "methodology": { approach, dataCollection, analysisMethod, limitations[] }
- "results": { findings[{label,value,description}], tables[...], charts[...] }
- "discussion": { interpretation, comparison, implications[], futureWork[] }
- "references": { items[{id, citation, doi}] }
- "layout": { style ("ieee"|"apa"|"chicago"), columns (1|2), showCharts, abstractStyle ("structured"|"narrative") }

```
Return JSON: { "message": "...", "sections": { ... } }
Only return changed sections.`
}
```

3-4. 생성 파이프라인

```
// api/reports/generate/route.ts (수정)

// Phase 1: 연구 컨텍스트 수집
const context = await fetchResearchContext(topic)
emit('section', { section: 'metadata', data: {
  title: `Research on ${topic}`,
  keywords: context.trends.topKeywords,
  authors: [], affiliation: ''
}})

// Phase 2: AI 초안 생성
const draft = await generatePaperDraft(topic, context)
for (const [section, data] of Object.entries(draft)) {
  emit('section', { section, data })
  await delay(200) // Progressive fill 효과
}
```

```

}

// Phase 3: DB 저장
await savePaper(reportId, fullData)
emit('done', {})

```

3-5. 템플릿 예시

템플릿	용도	특징
IEEE	공학/컴퓨터 학술지	2단 레이아웃, 작은 폰트, 줄간격 좁음
APA	사회과학 학술지	1단, 넓은 마진, Times New Roman
Presentation	발표용 요약	큰 폰트, 차트 강조, 핵심만
Draft	초안 검토용	줄번호, 넓은 줄간격, 코멘트 공간

```

// app/components/templates/IEEETemplate.tsx

export default function IEEETemplate({ data, sectionUpdates }: TemplateProps) {
  const { metadata, abstract, introduction, methodology,
    results, discussion, references, layout } = useSections(data, sectionUpdates)
  const abstractStyle = layout?.abstractStyle ?? 'structured'

  return (
    <div className="max-w-4xl mx-auto bg-white text-gray-900 font-serif text-[10px] leading-tight">
      {/* Title Block */}
      <div className="text-center py-6">
        <h1 className="text-xl font-bold">{metadata?.title}</h1>
        <p className="text-sm mt-2 italic">
          {metadata?.authors?.join(', ')}
        </p>
        <p className="text-xs text-gray-500">{metadata?.affiliation}</p>
      </div>

      {/* Abstract */}
      <div className="columns-2 gap-6">
        <div className="mb-4">
          <h2 className="font-bold italic">Abstract</h2>
          {abstractStyle === 'structured' ? (
            <>
              <p><b>Background:</b> {abstract?.background}</p>
              <p><b>Objective:</b> {abstract?.objective}</p>
              <p><b>Method:</b> {abstract?.method}</p>
              <p><b>Results:</b> {abstract?.results}</p>
              <p><b>Conclusion:</b> {abstract?.conclusion}</p>
            </>
          ) : (
            <p>{abstract?.background} {abstract?.objective} ...</p>
          )}
        </div>
        {/* ... 나머지 섹션 ... */}
      </div>
    </div>
  )
}

```

```
)  
}
```

3-6. 채팅 활용 예시

사용자: "abstract를 서술형으로 바꿔줘"
→ AI: layout.abstractStyle = 'narrative' 수정

사용자: "methodology에 실험 환경 추가해줘. GPU는 A100 4장"
→ AI: methodology.dataCollection에 실험 환경 텍스트 추가

사용자: "참고문헌에 Vaswani 2017 Attention 논문 추가"
→ AI: references.items에 항목 추가

사용자: "results 차트 숨기고 테이블만 보여줘"
→ AI: layout.showCharts = false

사용자: "IEEE 포맷에서 APA로 바꿔줘"
→ 템플릿 스위처로 전환 (데이터 동일, 렌더링만 변경)

4. 다른 활용 사례

이 프레임워크는 "구조화된 문서 + AI 생성 + 반복 수정" 패턴이면 무엇이든 적용 가능합니다.

사례	Data Fetcher	섹션 구조	템플릿
주식 리포트 (현재)	Yahoo Finance	header, valuation, thesis, verdict	Newspaper, Modern, Executive, Compact
논문 작성	Semantic Scholar	metadata, abstract, methodology, results	IEEE, APA, Presentation, Draft
사업계획서	시장 데이터 API	executive_summary, market, product, financials	Investor, Bank, Internal
이력서	LinkedIn API	profile, experience, skills, education	Modern, Classic, Creative
제품 스펙	Jira/Linear API	overview, requirements, architecture, timeline	PRD, Engineering, Executive
부동산 분석	네이버 부동산	property, market, valuation, recommendation	Agent, Investor, Comparison

교체가 필요한 부분 (용도별)

✅ 그대로 사용 가능:

- 폴더/리포트 관리 UI (FolderTree, drag-and-drop)

- 버전 관리 (save/restore/history)

- AI 채팅 리파인 루프 (FloatingChat → API → deep merge)

- PDF 내보내기 (html2canvas → jsPDF)

- Progressive Fill UX (SSE → 섹션별 애니메이션)

- 템플릿 스위칭 (VersionBar 드롭다운)

🔧 수정 필요:

- lib/types.ts → 섹션 인터페이스 재정의

- lib/prompts.ts → AI 프롬프트 수정
- lib/market-data.ts → Data Fetcher 교체
- app/components/templates/*.tsx → 렌더링 템플릿
- shared.tsx → useSections() 반환값, TemplateId 목록

❌ 교체 불필요:

- app/page.tsx (메인 레이아웃)
- app/api/chat/route.ts (deep merge 로직)
- app/api/reports/[id]/save|restore (버전 관리)
- app/api/folders/* (폴더 CRUD)
- lib/ai.ts (AI 추상화 레이어)
- lib/supabase.ts (DB 클라이언트)

5. 기술 스택 요약

레이어	기술	역할
Frontend	Next.js 16, React 19, Tailwind 4	앱 프레임워크
Animation	Framer Motion	Progressive Fill 효과
Charts	Recharts	데이터 시각화
PDF	html2canvas-pro + jsPDF	내보내기
DB	Supabase (Postgres)	리포트/폴더/채팅/버전 저장
AI (로컬)	Claude CLI	고품질 생성
AI (배포)	Gemini 2.5 Flash	서버리스 환경용
배포	Vercel	호스팅 + 서버리스 함수

6. 환경 변수

```
# 필수
NEXT_PUBLIC_SUPABASE_URL=...      # Supabase 프로젝트 URL
SUPABASE_SECRET_KEY=...          # service_role 키

# AI (택 1)
AI_PROVIDER=claude                # 로컬: Claude CLI
AI_PROVIDER=gemini                # Vercel: Gemini SDK
GEMINI_API_KEY=...                # Gemini 사용 시 필요
```

7. DB 스키마

```
-- 리포트 (용도에 관계없이 동일 구조)
CREATE TABLE reports (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title TEXT NOT NULL,
  ticker TEXT NOT NULL,           -- 또는 topic, subject 등으로 활용
  folder_id UUID REFERENCES report_folders(id),
  status TEXT DEFAULT 'generating',
```

```

data JSONB DEFAULT '{}',          -- ← 섹션 데이터 전체 (자유 구조)
version INTEGER DEFAULT 1,
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now()
);

-- 버전 스냅샷
CREATE TABLE report_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  report_id UUID REFERENCES reports(id) ON DELETE CASCADE,
  version INTEGER NOT NULL,
  data JSONB NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 채팅 히스토리
CREATE TABLE report_chats (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  report_id UUID REFERENCES reports(id) ON DELETE CASCADE,
  role TEXT NOT NULL,          -- 'user' | 'assistant'
  content TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 폴더
CREATE TABLE report_folders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  parent_id UUID REFERENCES report_folders(id),
  sort_order INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT now()
);

```

data 컬럼이 JSONB이므로 어떤 섹션 구조든 스키마 변경 없이 저장 가능합니다.